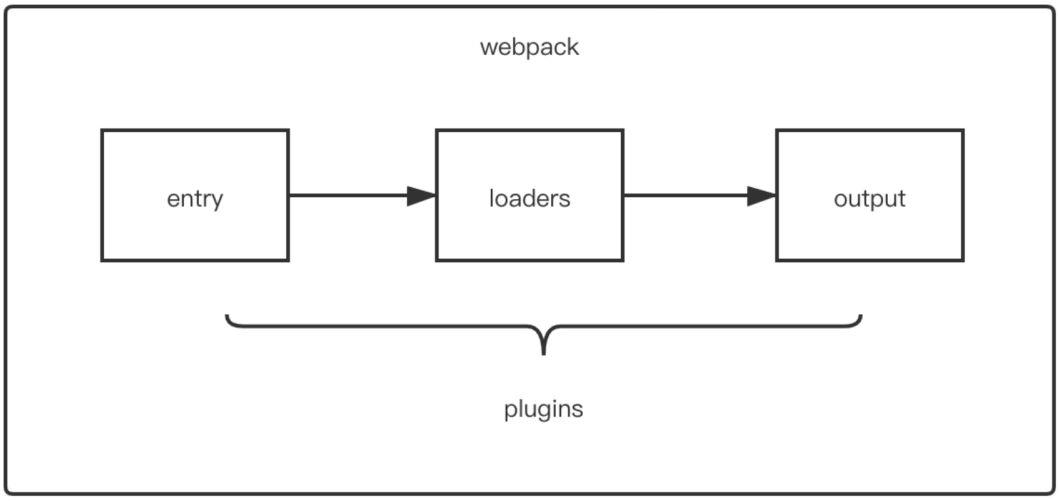
- webpack 打包原理
  - 寻找入口文件
  - 分析入口文件，获取所有的依赖
  - 读取多有的依赖文件并进行代码的转化

- webpack 中的 plugin 和 loader 的区别



  - loader主要拓展 webpack 对其他类型文件的处理能力
    - 原生 webpack 只能处理 js 文件，通常会使用 style-loader、css-loader、ts-loader 等loader 来拓展 webpack 对其他类型文件的处理能力
  - 用来拓展 webpack 的功能，插件可运行在 webpack 的不同生命周期
    - 用于打包优化、资源管理、注入环境变量等

- webpack手写过 plugin 和 loader 吗?
  - 虽然工作里没有接触过手写 plugin、loader 的需求，但是我在私下里做技术提升的时候自己写过简单的 plugin、loader，如果未来工作中需要手写 plugin、loader，是完全可以应对的

- webpack 打包体积如何优化?
- webpack 打包速度如何优化?

- 重绘与回流（重排）
  - 回流（重排）：当render tree（渲染树）中的一部分(或全部)因为元素的规模尺寸，布局，隐藏等改变而需要重新构建
    - 回流必将引起重绘
  - 重绘：当render tree（渲染树）中的一些元素需要更新属性，而这些属性只是影响元素的外观，风格，而不会影响布局的，比如background-color

- 设计框架思路
  - UI 模板复用

    - 高阶组件逻辑复用
    - 接口简约、渐进式适配
    - 封装平台方法以便于开发、调试
    - 封装全局方法以便于调试
    - 制定开发规范解决代码低质问题
        - 引入eslint、commit
    - React 性能优化
    - 老版本生命周期的适配
    - 脚手架

- V8引擎的垃圾回收机制
    - https://blog.csdn.net/qq_43477721/article/details/116699530

- 解决跨域问题
    - $.ajax解决跨域：dataType: 'jsonp'
        - 如此设置，$ajax方法内部则不会使用XMLHttpRequest对象请求，使用其他方式
    - 标签中的src、href属性具备跨域请求资源的能力
        - jsonp本质是使用script标签的src属性跨域的能力
        - jsonp只能发送get请求
    - 跨域资源共享
        - 在后端设置允许访问的域名

- React性能优化
    - 渲染
        - shouldComponentUpdate
        - React.memo()
        - PureComponent
    - useEffect只关联指定参数
    - 不在render中写过多逻辑
    - 绑定方法要尽量写在外面，尽量不写在JSX中

- 性能优化
    - 加载更快
        - 让传输的数据包更小（压缩文件/图片）：图片压缩和文件压缩
        - 减少网络请求的次数：雪碧图/精灵图、节流防抖
        - 减少渲染的次数：缓存（HTTP缓存、本地缓存、Vue的keep-alive缓存等）
    - 渲染更快的方法：
        - 提前渲染：ssr服务器端渲染

        - 避免渲染阻塞：CSS放在HTML的head中 JS放在HTML的body底部
        - 避免无用渲染：懒加载
        - 减少渲染次数：对dom查询进行缓存、将dom操作合并、使用减少重排的标签

- 性能优化
    - 技术
        - 精灵图减少请求次数
        - CND
        - 骨架图
        - 离线包
        - 服务端渲染
        - 减少首屏加载的资源
            - 图片压缩
        - 网络
            - 使用 http2.0（2进制传输）
            - 防抖、节流
        - 代码的冗余程度
            - 逻辑复用、封装模板
    - 代码
        - React
            - 渲染
                - shouldComponentUpdate
                - React.memo()
                - PureComponent
            - useEffect只关联指定参数
            - 不在render中写过多逻辑
            - 绑定方法要尽量写在外面，尽量不写在JSX中
            - 组件懒加载
        - JS
            - 少写闭包，减少防止内存泄露
            - link标签写在上边，script标签写在下面
            - 减少变量取值的次数
            - 避免重排、重绘
        - webpack
            - 分离css
            - 压缩体积
        -

    - 举例
        - 夸克书城改版
        - 轮播信息
        - 用友设计的架构

- var、let、const的区别
    - var
        - 可重复定义
        - 变量声明提升
        - 挂载到window对象上
        - 作用域只有全局或函数
    - let
        - 不可重复定义
        - 无变量声明提升
        - 不会挂载到window上
        - 会和大括号形成块级作用域
    - const
        - 与let类似，但是声明必须赋值，且赋值后不可修改

- 箭头函数和普通函数的区别
    - 箭头函数无法创建对象，只能当做函数来使用
    - 箭头函数可以同时省略 {} 和 return
    - 箭头函数没有arguments
    - 箭头函数无this，其this值取决于最近外层的非箭头函数
    - 箭头函数的参数不能重复命名
    - 箭头函数不能用call、bind、apply改变this指向

- 说一说 HashRouter 和 HistoryRouter 的区别和原理
    - history 路由没有“#”，hash 有
    - history 利用浏览历史记录栈的 api 来实现，hash 通过监听 location 中的 hash 值变化来实现
    - 相同的 url，history 会触发添加到浏览器的记录栈中，hash 不会触发
    - history 需要后端配合，hash 不需要

- 各种模块化引入方式之间的区别?

- 模块化导入的方式及区别?
    - ES6

```
//node_modules/myStore/index.js
let name = "sss";
export { name };
------------------------------
import { name as name2 } from "myStore";
```

- commomJS

```
//node_modules/myStore/index.js
let name = "sss";
module.exports = {
  name
};
------------------------------
let { name: name3 } = require("myStore");
```

- 区别
  - CommonJS是运行时加载，因为它导出的是一个对象，对象只有在运行的时候才会创建，ES6在编写的时候就已经确定了模块间的依赖关系
  - CommonJS导出的是值拷贝，导出后CommonJS内部的值改变不会影响外部，除非重新加载，而ES6模块导出的是值引用，内部的值改变会影响
  - CommonJS使用require导入，ES6通过[import from]的形式
  - CommonJS的this指向当前的模块，ES6的this指向undefined，因为它使用的是严格模式
- 代码实测
  - 引用值在导入后修改，会影响其他导入
  - 原始导出的地方也会收到影响
  - 基本值在导入后修改，也会影响其他导入
  - 原始导出的地方不会收到影响

- type 和 interface 的区别?
- 解释一下 ts 中泛型的概念及使用场景?
- 发布订阅模式和观察者模式的区别?
- Vue 中 computed 和 watch 的区别?